

- Triplet `[2, 4, 8]` occurs at indexes `(0, 3, 4)`, `(0, 3, 5)`, `(2, 3, 4)`, `(2, 3, 5)`.
- Triplet `[1, 2, 4]` occurs at indexes `(1, 2, 3)`.

Intuition

For a triplet to form a geometric sequence, it has to adhere to two main rules:

1. It consists of three values that follow a geometric sequence with a common ratio r .
2. The three values forming the triplet must appear in the same order within the array as they do in the geometric sequence. This means for a geometric triplet $(\text{nums}[i], \text{nums}[j], \text{nums}[k])$, the indexes must follow the order $i < j < k$.

How can we represent a geometric sequence so that it follows rule 1? Let's say the first number is x . The second number is the first number multiplied by r (i.e., $x \cdot r$), and the third is the second number multiplied by r (i.e., $x \cdot r \cdot r = x \cdot r^2$). So, a triplet in a geometric sequence can be represented as $(x, x \cdot r, x \cdot r^2)$.

A brute force approach is to iterate over every possible triplet in the array and check if any of these triplets follow a geometric progression. However, it would take three nested for-loops to search through all the triplets, resulting in a time complexity of $O(n^3)$, where n denotes the length of the input array. Can we do better?

An important observation here is that if we know one value of a triplet, we can calculate what the other two values should be.

This is because all three values are related by the common ratio r . So, for any number x in the array, we just need to find the values $x \cdot r$ and $x \cdot r^2$ to form a geometric triplet $(x, x \cdot r, x \cdot r^2)$. However, we could run into issues when using this triplet representation. While it's clear the values $x \cdot r$ and $x \cdot r^2$ must be positioned to the right of x , we have to be careful since the order of these values matters: we don't want to accidentally identify a triplet such as $(x, x \cdot r^2, x \cdot r)$, which is invalid:

$$\begin{array}{ccccccc} & & x & & & & \\ & & \downarrow & & & & \\ [& \cdot & \cdot & x & \cdot & \cdot & x \cdot r^2 & \cdot & x \cdot r &] \\ \text{triplet values } (x, x \cdot r^2, x \cdot r) & \text{appear in the wrong order} \end{array}$$

We can work around this issue by using the $(x/r, x, x \cdot r)$ triplet representation, which allows us to always maintain order by looking for x/r to the left of x and $x \cdot r$ to the right:

$$\left[\underbrace{\cdot \quad \cdot \quad \cdot \quad \cdot}_{x/r} \right] \times \left[\underbrace{\cdot \quad \cdot \quad \cdot}_{x \cdot r} \right]$$

find x/r find $x \cdot r$

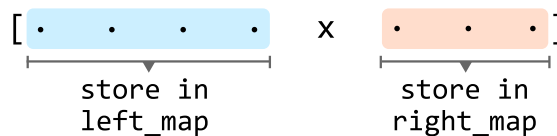
One way we can find the x/r and $x \cdot r$ values is by linearly searching through the left and right subarrays. This linear search would need to be done for each number in the array, resulting in an $O(n^2)$ time complexity. While this is an improvement from the brute force solution, it would be great if we had a way to find those values faster.

Hash maps

A hash map would be a great way to solve this problem, as it allows us to query specific values in constant time.

What we would need are two hash maps:

- A hash map that contains numbers to the left of each x (**left_map**).
- A hash map that contains numbers to the right of each x (**right_map**).



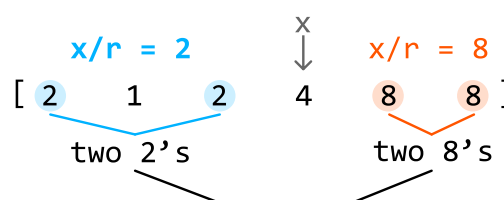
Hash maps allow us to query for both x/r in the left hash map and query for $x \cdot r$ in the right hash map in constant time on average. Note that a hash map would be preferred over a hash set because hash maps can also store the frequency of each value it stores. This is crucial since the array might contain duplicates, and we need to know the frequency of each value to accurately identify all possible triplets.

Finding all $(x/r, x, x \cdot r)$ triplets

Our goal is to find all triplets that follow a geometric sequence, representing each number in the array as the middle (x) number of a triplet.

Before we find a triplet's x/r value, we need to **check if x is divisible by r** . If it's not, it's impossible to form a triplet from the current value of x . Otherwise, we can proceed to look for the triplet.

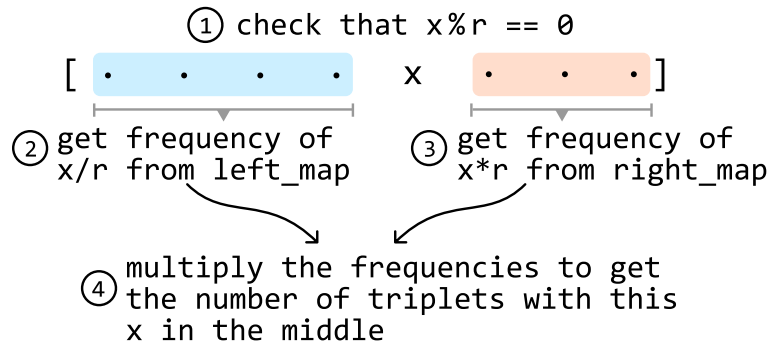
For any element x , there could be multiple instances of x/r in **left_map** and multiple instances of $x \cdot r$ in **right_map**, implying that multiple triplets can be formed using x as the middle value. So, to get the total number of triplets that can be formed with x in the middle, **multiply the frequencies of $x \cdot r$ and x/r** :



$$2 * 2 = 4$$

→ 4 instances of [2, 4, 8]

This overall methodology can be summarized in the following steps:



Note that if either x/r or $x*r$ are not found in their hash maps, their frequency is 0 by default.

Let's implement this strategy using the example below:

$$[2 \quad 1 \quad 2 \quad 4 \quad 8 \quad 8], \quad r = 2$$

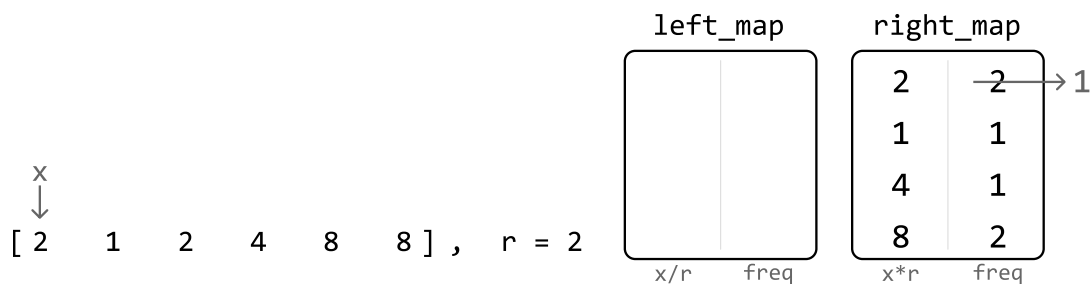
To ensure the hash maps always contain the correct values, we'd need to incorporate a dynamic strategy that involves updating the hash maps as we go because the values in both hash maps will be different depending on the position of x in the array.

Since we're traversing the array from left to right, we should **initially fill the right hash map with all values in the array**. This is because, before the start of the iteration, every element is a potential candidate for $x*r$. Meanwhile, the left hash map is initially empty because there are no preceding elements to consider as potential x/r values:

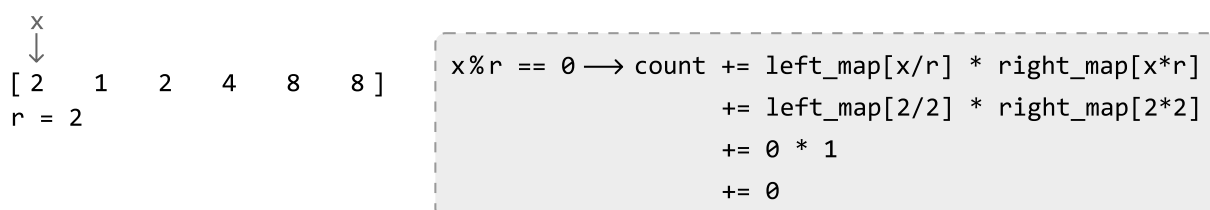
						left_map	right_map
							2
							1
							4
							8
							2
						x/r	x*r
						freq	freq

Now let's look for triplets. Start by representing the first value as the middle value (x) of a triplet.

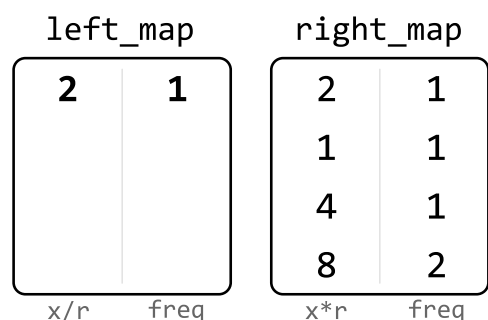
First, let's **update right_map**. We should remove the current value (2) from **right_map** since this 2 is not to the right of itself. There are two 2's in **right_map**, so let's reduce its frequency to 1:



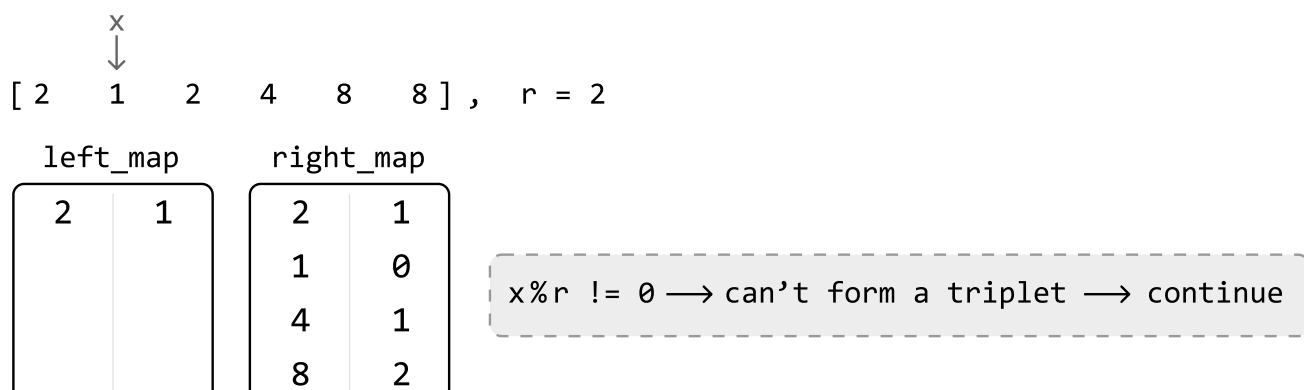
Next, **check if x/r is an integer**. In this case, it is, so let's **find the number of triplets with x as the middle number** by multiplying the frequencies of x/r and $x \cdot r$, which we can get from the respective hash maps. Since **left_map** doesn't contain x/r at this point, its frequency is 0:



Before moving on to the next value, let's **add the current number to the left_map** because it now becomes a potential x/r value for future triplets in the array:



Repeating this process for the rest of the array allows us to find all geometric triplets with a ratio of r . To clarify, the hash maps in the upcoming diagrams represent their state at the current position of x in the array. This means that **left_map** includes values to the left of the current x , and the **right_map** includes values to the right of it.



$\overbrace{x/r} \quad \overbrace{\text{freq}}$ $\overbrace{x*r} \quad \overbrace{\text{freq}}$

$\begin{matrix} & & x \\ & & \downarrow \\ [2 & 1 & 2 & 4 & 8 & 8], & r = 2 \end{matrix}$

left_map		right_map	
2	1	2	0
1	1	1	0
		4	1
		8	2
x/r	freq	$x*r$	freq

```

x%r == 0 → count += left_map[x/r] * right_map[x*r]
                += left_map[2/2] * right_map[2*2]
                += 1 * 1
                += 1

```

$\begin{matrix} & & x \\ & & \downarrow \\ [2 & 1 & 2 & 4 & 8 & 8], & r = 2 \end{matrix}$

left_map		right_map	
2	2	2	0
1	1	1	0
		4	0
		8	2
x/r	freq	$x*r$	freq

```

x%r == 0 → count += left_map[x/r] * right_map[x*r]
                += left_map[4/2] * right_map[4*2]
                += 2 * 2
                += 4

```

$\begin{matrix} & & x \\ & & \downarrow \\ [2 & 1 & 2 & 4 & 8 & 8], & r = 2 \end{matrix}$

left_map		right_map	
2	2	2	0
1	1	1	0
4	1	4	0
		8	1
x/r	freq	$x*r$	freq

```

x%r == 0 → count += left_map[x/r] * right_map[x*r]
                += left_map[8/2] * right_map[8*2]
                += 1 * 0
                += 0

```

$\begin{matrix} & & x \\ & & \downarrow \\ [2 & 1 & 2 & 4 & 8 & 8], & r = 2 \end{matrix}$

left_map		right_map	
2	2	2	0
1	1	1	0
.	.	.	.
x/r	freq	$x*r$	freq

```

x%r == 0 → count += left_map[x/r] * right_map[x*r]
                += left_map[8/2] * right_map[8*2]

```

4	1	4	0
8	1	8	0
x/r	freq	x*r	freq

+= 1 * 0

+= 0

Implementation

Python

JavaScript

Java

```

from typing import List
from collections import defaultdict

def geometric_sequence_triplets(nums: List[int], r: int) → int:
    # Use 'defaultdict' to ensure the default value of 0 is returned when
    # accessing a key that doesn't exist in the hash map. This effectively sets
    # the default frequency of all elements to 0.
    left_map = defaultdict(int)
    right_map = defaultdict(int)
    count = 0
    # Populate 'right_map' with the frequency of each element in the array.
    for x in nums:
        right_map[x] += 1
    # Search for geometric triplets that have x as the center.
    for x in nums:
        # Decrement the frequency of x in 'right_map' since x is now being
        # processed and is no longer to the right.
        right_map[x] -= 1
        if x % r == 0:
            count += left_map[x // r] * right_map[x * r]
        # Increment the frequency of x in 'left_map' since it'll be a part of the
        # left side of the array once we iterate to the next value of 'x'.
        left_map[x] += 1
    return count

```

Complexity Analysis

Time complexity: The time complexity of `geometric_sequence_triplets` is $O(n)$ because we iterate through the `nums` array and perform constant-time hash map operations at each iteration.

Space complexity: The space complexity is $O(n)$ because the hash maps can grow up to n in size.

☐ Mark as Complete

